

HW3_Problem2

January 28, 2026

1 Problem 2

This problem focuses on supervised learning concepts using Ensemble Models, utilizing popular learning frameworks. Additionally, the final question benchmarks the performance of a semi-supervised learning approach using the scikit-learn library. Please fix the random state to 42 where required.

```
[ ]: ## Collection of imports for Problem 2
    # import pandas as pd
    # import numpy as np
    # from sklearn.model_selection import train_test_split, StratifiedShuffleSplit
    # from sklearn.tree import DecisionTreeClassifier
    # from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
    # from sklearn.svm import SVC
    # from sklearn.ensemble import BaggingClassifier
    # from sklearn.metrics import classification_report, confusion_matrix
    # from sklearn.semi_supervised import SelfTrainingClassifier
```

1.1 2.1

1.1.1 Load the provided dataset, pima.csv, print its shape and display the distribution of the target variable.

```
[ ]: import pandas as pd
    # Downloaded the dataset from Kaggle and renamed from diabetes.csv to pima.csv, ↵
    ↪ in order to allign with the request.
    data = pd.read_csv('pima.csv')
    data_og=pd.read_csv('pima.csv') # Keep a copy of the original dataset
```

Print the first ten lines to take a peek in the data

```
[ ]: data.head(10)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	
5	5	116	74	0	0	25.6	

6	3	78	50	32	88	31.0
7	10	115	0	0	0	35.3
8	2	197	70	45	543	30.5
9	8	125	96	0	0	0.0

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1
5	0.201	30	0
6	0.248	26	1
7	0.134	29	0
8	0.158	53	1
9	0.232	54	1

Print the shape of the dataset

```
[ ]: print("Dataset shape:", data.shape)
```

Dataset shape: (768, 9)

Extract the target variable

```
[ ]: target=data['Outcome']
```

Print the target variable distribution

```
[ ]: print("Target Variable Distribution:")
print(target.value_counts())
```

Target Variable Distribution:

Outcome

0 500

1 268

Name: count, dtype: int64

Print the percentage distribution of the target variable

```
[ ]: print((target.value_counts()/data.shape[0] * 100).round(2).astype(str) + '%')
```

Outcome

0 65.1%

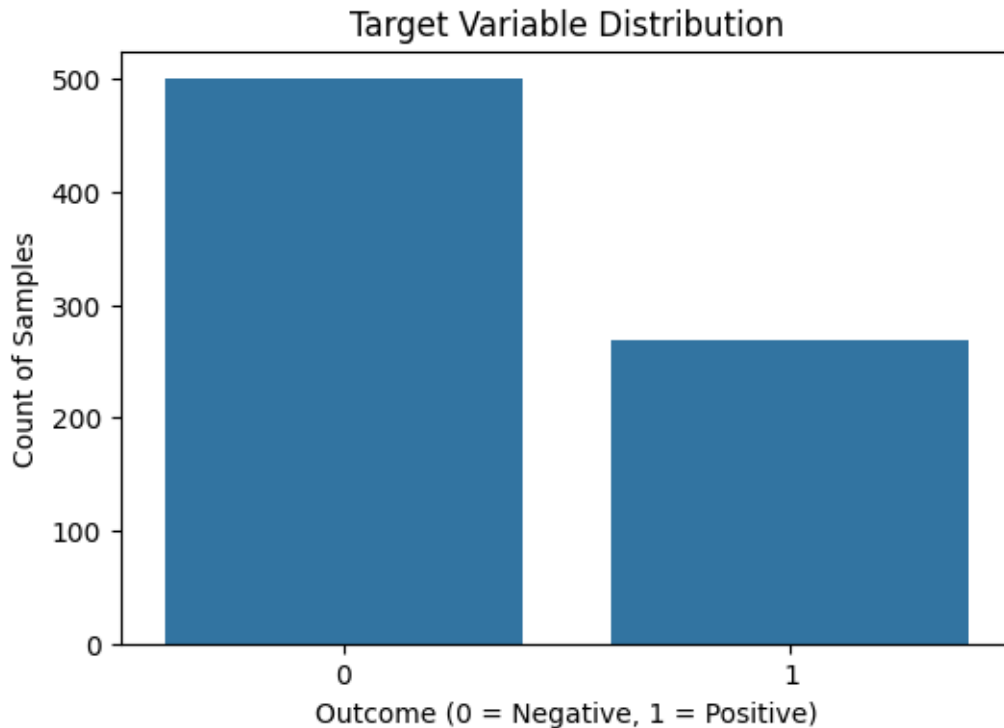
1 34.9%

Name: count, dtype: object

Plot the distribution of the target variable

```
[ ]: from matplotlib import pyplot as plt
import seaborn as sns
```

```
plt.figure(figsize=(6,4))
sns.countplot(x=target, data=data)
plt.title("Target Variable Distribution")
plt.xlabel("Outcome (0 = Negative, 1 = Positive)")
plt.ylabel("Count of Samples")
plt.show()
```



1.2 2.2

1.2.1 Display the main statistical characteristics of each column in the data in a tabular format. Identify columns where the minimum value takes an unreasonable value equal to zero. Replace these zero values with the median of the corresponding column, as they represent missing values based on the dataset's creation protocol. Print the updated dataset's characteristics in a tabular format and list the columns affected by this step.

Display the summary statistics of the dataset.

```
[ ]: data.describe()
```

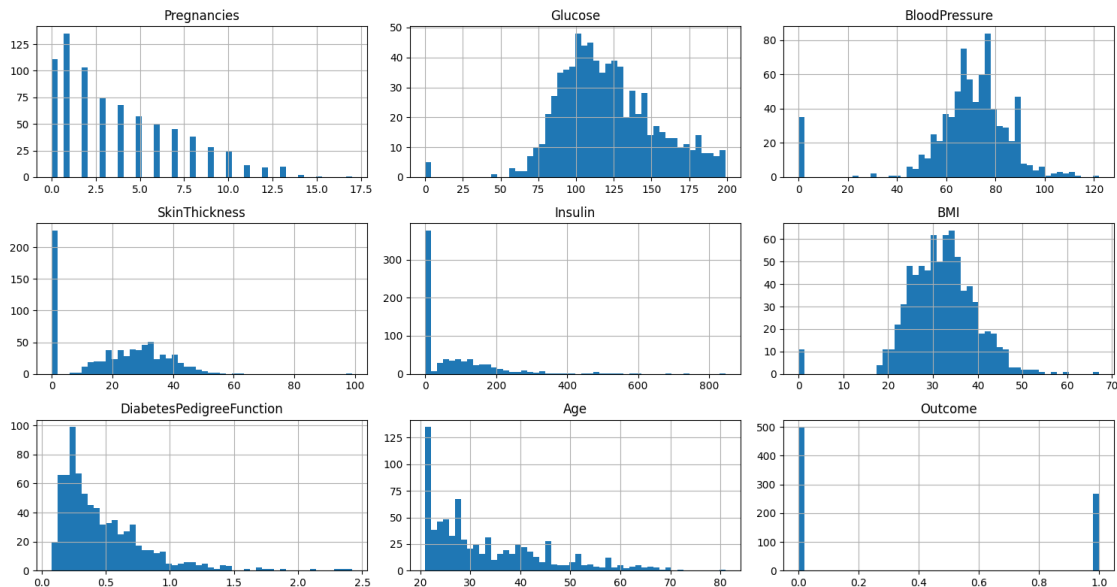
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin \
count	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479
std	3.369578	31.972618	19.355807	15.952218	115.244002

min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000
75%	6.000000	140.250000	80.000000	32.000000	127.250000
max	17.000000	199.000000	122.000000	99.000000	846.000000

	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000
mean	31.992578	0.471876	33.240885	0.348958
std	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.078000	21.000000	0.000000
25%	27.300000	0.243750	24.000000	0.000000
50%	32.000000	0.372500	29.000000	0.000000
75%	36.600000	0.626250	41.000000	1.000000
max	67.100000	2.420000	81.000000	1.000000

Plot histograms of all features to visually identify any abnormalities.

```
[ ]: data.hist(bins=50, figsize=(15,8)) # Create the histograms, with 50 bins
plt.tight_layout() # Use tight_layout so that the histogram title does not
    ↪ collide with the x-axis ticks/values.
plt.show()
```



We wouldn't expect the attributes 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin' and 'BMI', to have zero values. These zeroes represent missing values.

Define the columns that should not contain zero values, and count the number of zeroes in each column.

```
[ ]: unreasonable_columns = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
print("Number of zero values in unreasonable columns:")
print((data[unreasonable_columns] == 0).sum())
```

Number of zero values in unreasonable columns:

```
Glucose      5
BloodPressure 35
SkinThickness 227
Insulin      374
BMI          11
```

dtype: int64

Replace all zero values in the unreasonable columns with the median of the non-zero values (In order to calculate the median of the actual values we remove the zeroes from the calculation, because not doing so, would alter the values' distribution). Then we replace those zeroes with the median.

```
[ ]: for col in unreasonable_columns:
    median_value = data[col][data[col] != 0].median()
    data[col] = data[col].replace(0, median_value)
```

Confirm there are no zero values remaining in the unreasonable columns.

```
[ ]: (data[unreasonable_columns]==0).any()
```

```
Glucose      False
BloodPressure False
SkinThickness False
Insulin      False
BMI          False
```

dtype: bool

Print the updated dataset's statistics

```
[ ]: data.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin \
count	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	121.656250	72.386719	29.108073	140.671875
std	3.369578	30.438286	12.096642	8.791221	86.383060
min	0.000000	44.000000	24.000000	7.000000	14.000000
25%	1.000000	99.750000	64.000000	25.000000	121.500000
50%	3.000000	117.000000	72.000000	29.000000	125.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000
max	17.000000	199.000000	122.000000	99.000000	846.000000

	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000
mean	32.455208	0.471876	33.240885	0.348958
std	6.875177	0.331329	11.760232	0.476951

min	18.200000	0.078000	21.000000	0.000000
25%	27.500000	0.243750	24.000000	0.000000
50%	32.300000	0.372500	29.000000	0.000000
75%	36.600000	0.626250	41.000000	1.000000
max	67.100000	2.420000	81.000000	1.000000

Identify and print the affected columns.

```
[ ]: affected_columns = []
      for col in unreasonable_columns:
          if not data_og[col].equals(data[col]):
              affected_columns.append(col)

      # Print affected columns
      print("Affected columns:")
      print(affected_columns)
```

Affected columns:

```
['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
```

1.3 2.3

1.3.1 Perform a stratified split of the dataset, allocating 700 instances to the training set and the remaining samples as the test set. Ensure that the target variable is the Outcome column, while the rest of the columns constitute the feature space for the classification task.

```
[ ]: X = data.drop(columns=['Outcome'])
      y = data['Outcome']
```

```
[ ]: from sklearn.model_selection import train_test_split

      X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                          train_size =700,
                                                          stratify = y,
                                                          random_state=42)
```

```
[ ]: print("Train set size:",len(X_train))
      print("Test set size:", len(X_test))
```

Train set size: 700

Test set size: 68

1.4 2.4

1.4.1 Train the following models:

1.4.2 - A simple Decision Tree classifier using the default values of its parameters.

1.4.3 - A Random Forest classifier using the default values of its parameters.

1.4.4 - A Bagging classifier with an SVM classifier (linear kernel) and 10 estimators.

1.4.5 - An AdaBoost classifier with a decision tree classifier, using 100 estimators and a learning rate of 0.25

1.4.6 For each model, display a classification report presenting the standard classification metrics, precision, recall, and f1-score and the confusion matrix as they were calculated on the test set.

1.4.7 Training

Decision Tree classifier

```
[ ]: from sklearn.tree import DecisionTreeClassifier

dt_clf = DecisionTreeClassifier(random_state=42)
dt_clf.fit(X_train, y_train)
y_pred_dt = dt_clf.predict(X_test)
```

Random Forest classifier

```
[ ]: from sklearn.ensemble import RandomForestClassifier

rf_clf = RandomForestClassifier(random_state=42)
rf_clf.fit(X_train, y_train)
y_pred_rf = rf_clf.predict(X_test)
```

Bagging classifier (SVM)

```
[ ]: from sklearn.ensemble import BaggingClassifier
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

bag_svm_pipeline = make_pipeline(StandardScaler(),
                                BaggingClassifier(SVC(kernel='linear',
                                                    random_state=42),
                                                    n_estimators=10,
                                                    random_state=42))

bag_svm_pipeline.fit(X_train, y_train)
y_pred_bag_svm = bag_svm_pipeline.predict(X_test)
```

AdaBoost classifier (Decision Tree)

```
[ ]: from sklearn.ensemble import AdaBoostClassifier

ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(random_state=42),
    n_estimators=100,
    learning_rate=0.25,
    algorithm="SAMME",
    random_state=42)
ada_clf.fit(X_train, y_train)
y_pred_ada = ada_clf.predict(X_test)
```

1.4.8 Evaluation

```
[ ]: import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix

def model_evaluation_and_conf_mat(model, X_test, y_test, model_name):
    """Evaluates the model on test data, and prints classification metrics
    along with a Seaborn heatmap confusion matrix."""
    print(f"\n {model_name} Classification metrics")
    y_pred = model.predict(X_test)

    print(classification_report(y_test, y_pred))

    # Compute confusion matrix
    cm = confusion_matrix(y_test, y_pred)

    # Plot confusion matrix using Seaborn
    plt.figure(figsize=(6,5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=np.
    unique(y_test), yticklabels=np.unique(y_test))
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.title(f"Confusion Matrix - {model_name}")
    plt.show()

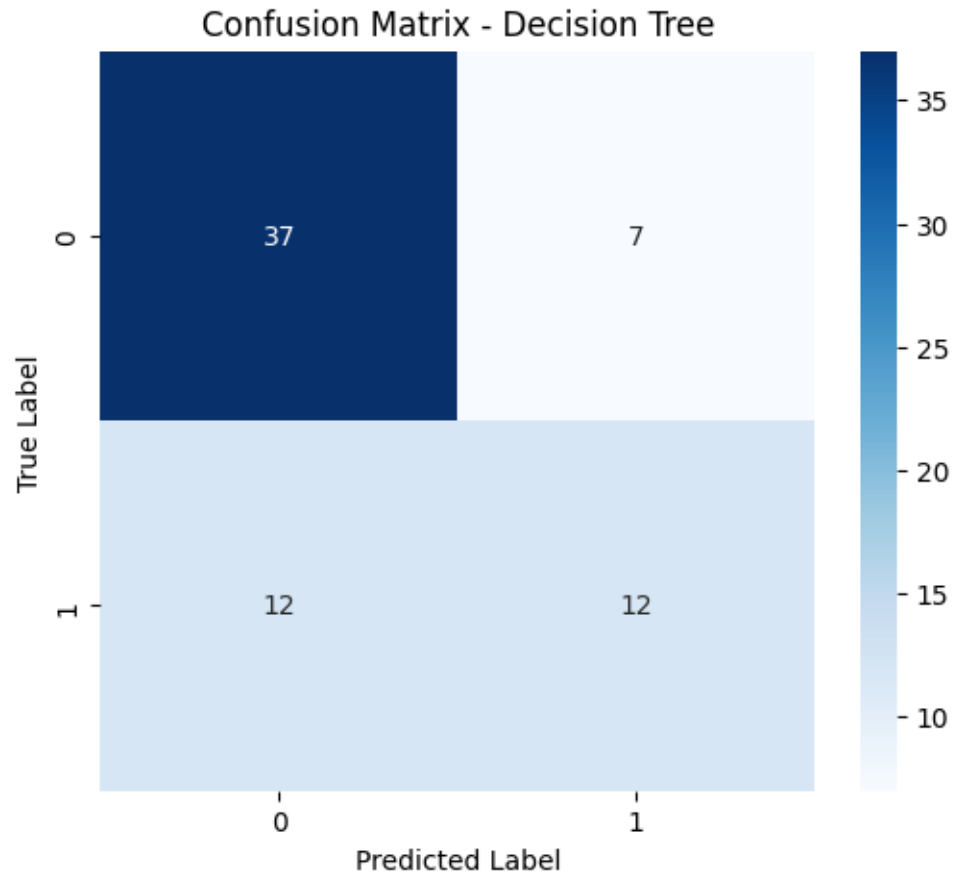
[ ]: print("\nDecision Tree Classification Report:")
model_evaluation_and_conf_mat(dt_clf, X_test, y_test, "Decision Tree")
```

Decision Tree Classification Report:

```
Decision Tree Classification metrics
      precision    recall  f1-score   support

0         0.76      0.84      0.80         44
```


	1	0.63	0.50	0.56	24
accuracy				0.72	68
macro avg		0.69	0.67	0.68	68
weighted avg		0.71	0.72	0.71	68

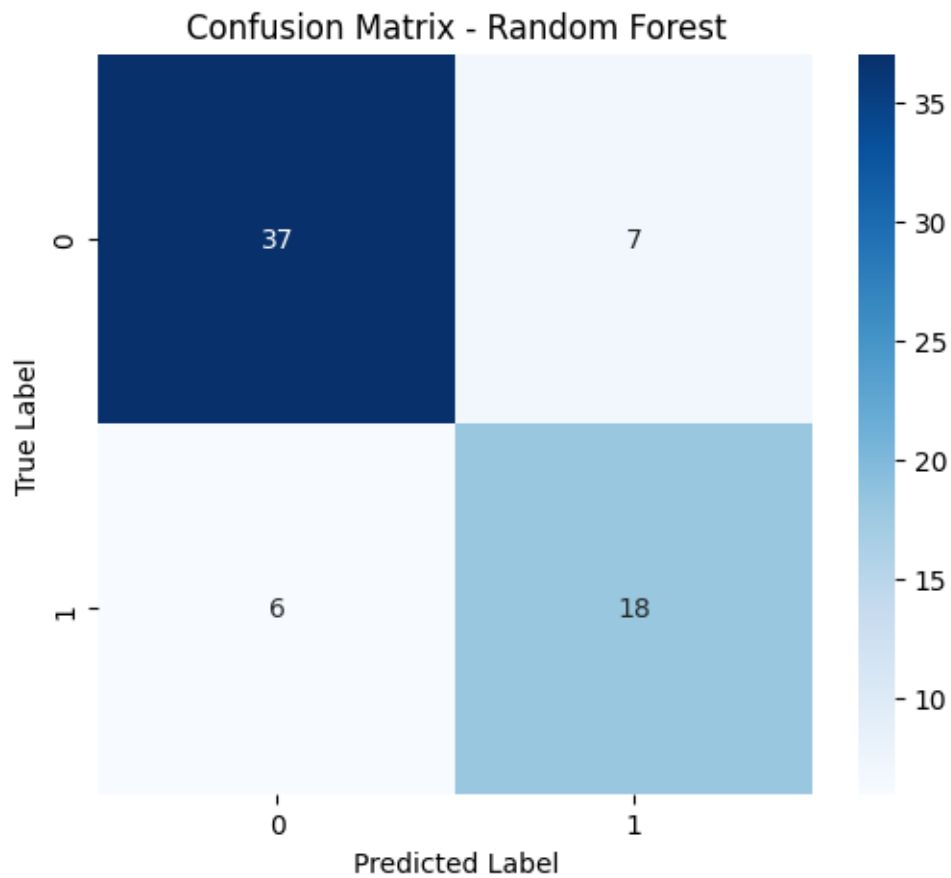


```
[ ]: print("\nRandom Forest Classification Report:")
model_evaluation_and_conf_mat(rf_clf, X_test, y_test, "Random Forest")
```

Random Forest Classification Report:

Random Forest Classification metrics				
	precision	recall	f1-score	support
0	0.86	0.84	0.85	44
1	0.72	0.75	0.73	24
accuracy			0.81	68

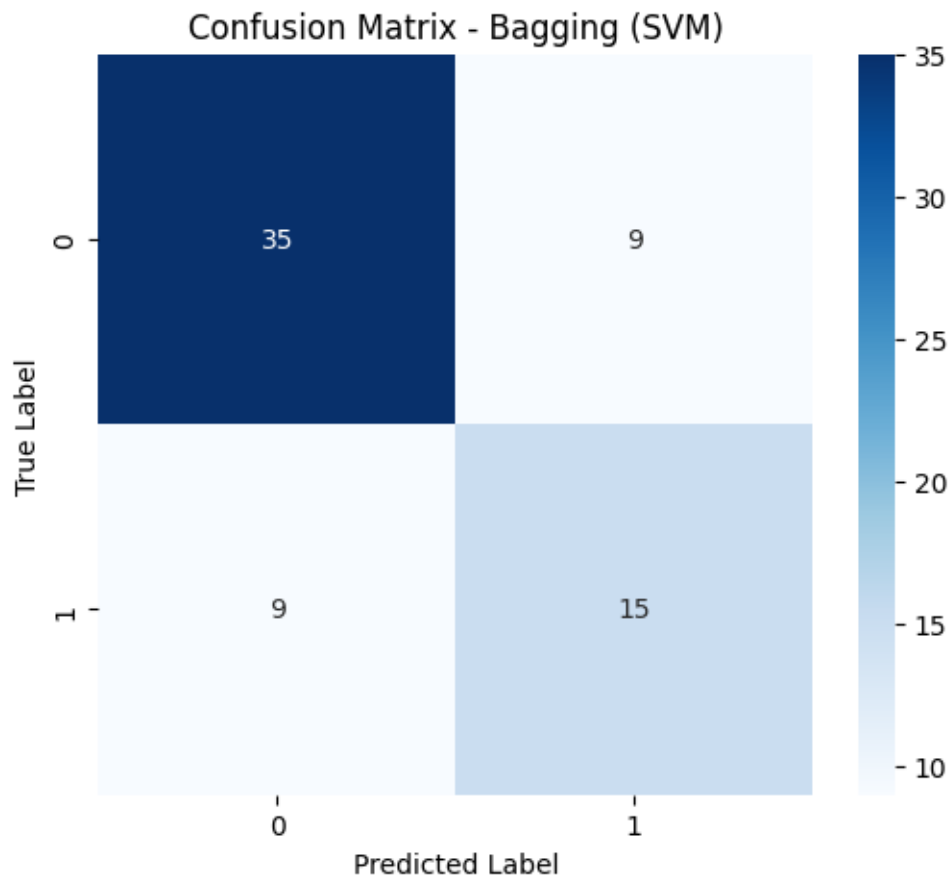
macro avg	0.79	0.80	0.79	68
weighted avg	0.81	0.81	0.81	68



```
[ ]: print("\nBagging with SVM Classification Report:")
      model_evaluation_and_conf_mat(bag_svm_pipeline, X_test, y_test, "Bagging (SVM)")
```

Bagging with SVM Classification Report:

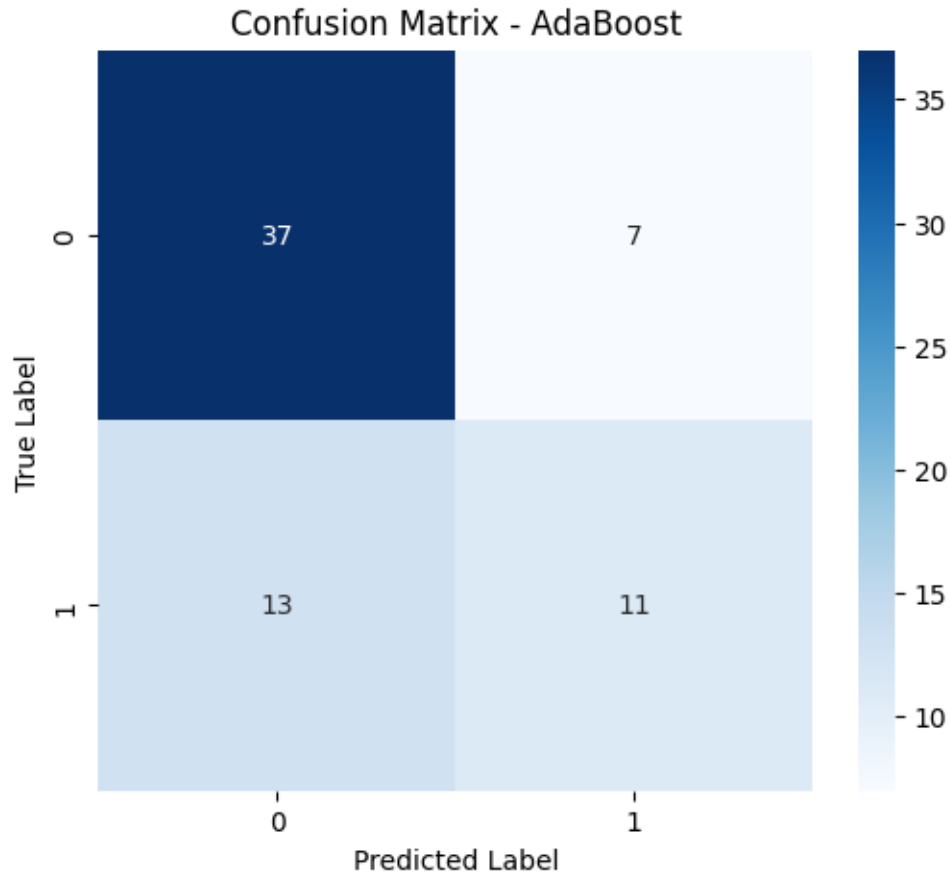
Bagging (SVM) Classification metrics				
	precision	recall	f1-score	support
0	0.80	0.80	0.80	44
1	0.62	0.62	0.62	24
accuracy			0.74	68
macro avg	0.71	0.71	0.71	68
weighted avg	0.74	0.74	0.74	68



```
[ ]: print("\nAdaBoost Classification Report:")
      model_evaluation_and_conf_mat(ada_clf, X_test, y_test, "AdaBoost")
```

AdaBoost Classification Report:

AdaBoost Classification metrics				
	precision	recall	f1-score	support
0	0.74	0.84	0.79	44
1	0.61	0.46	0.52	24
accuracy			0.71	68
macro avg	0.68	0.65	0.66	68
weighted avg	0.69	0.71	0.69	68



1.5 2.5

1.5.1 Compare the performance of the previous four models. Comment on the observed performance differences.

```
[ ]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
def model_metrics(model, X_test, y_test):
    y_pred = model.predict(X_test)
    return {"Accuracy": accuracy_score(y_test, y_pred),
            "Precision": precision_score(y_test, y_pred),
            "Recall": recall_score(y_test, y_pred),
            "F1-score": f1_score(y_test, y_pred)}
```

```
[ ]: metrics_df = pd.DataFrame([
    model_metrics(dt_clf, X_test, y_test),
    model_metrics(rf_clf, X_test, y_test),
    model_metrics(bag_svm_pipeline, X_test, y_test),
```

```

    model_metrics(ada_clf, X_test, y_test)
], index=["Decision Tree", "Random Forest", "Bagging (SVM)", "AdaBoost"]).
↳round(2).astype(str)

```

metrics_df

	Accuracy	Precision	Recall	F1-score
Decision Tree	0.72	0.63	0.5	0.56
Random Forest	0.81	0.72	0.75	0.73
Bagging (SVM)	0.74	0.62	0.62	0.62
AdaBoost	0.71	0.61	0.46	0.52

Looking at the results, Random Forest clearly stands out as the best model, achieving the highest accuracy (0.81) and the best balance between precision, recall, and F1-score. This shows how powerful ensemble methods can be in improving predictions and handling complex patterns in the data.

Bagging with SVM does a decent job too, with an accuracy of 0.74 and an F1-score of 0.62. It offers some improvements over a standalone model but doesn't quite reach the level of Random Forest.

On the other hand, the Decision Tree struggles a bit more. While its accuracy (0.72) isn't too far behind, its recall is quite low (0.50), meaning it misses a lot of positive cases. This suggests it's not as reliable when it comes to generalizing well.

AdaBoost, despite being another ensemble method, surprisingly performs the worst. Its recall (0.46) is the lowest, and its F1-score (0.52) reflects that it's having a hard time maintaining a good balance between precision and recall.

Overall, Random Forest is the clear winner, showing why it's such a popular choice.

1.6 2.6

1.6.1 Select the best-performing classifier from Question 4 as the base model. Wrap it in an instance of the `SelfTrainingClassifier` class with the arguments `criterion='threshold'` and `threshold=0.99`. Randomly select 200 instances from the initial training set as labeled data, marking the rest as unlabeled. Train the semi-supervised model on the combined labeled and unlabeled data and train the supervised model on only the labeled data. Evaluate both models on the same test set, display the classification reports, and add your comments.

```
[ ]: from sklearn.ensemble import RandomForestClassifier
```

```
base_model = RandomForestClassifier(random_state=42)
```

```
[ ]: from sklearn.semi_supervised import SelfTrainingClassifier
self_training_clf = SelfTrainingClassifier(base_model, criterion="threshold",
↳threshold=0.99)
```

```
[ ]: train_idx = X_train.index.to_numpy()
```

```
# Randomly select 200 indices from X_train's index
np.random.seed(42)
labeled_idx = np.random.choice(train_idx, 200, replace=False)
labeled_idx[:10] # Check the first 10 indices
```

```
array([748, 686, 408, 282, 423, 62, 331, 506, 284, 306], dtype=int64)
```

```
[ ]: X_labeled = X_train.loc[labeled_idx]
     y_labeled = y_train.loc[labeled_idx]
```

```
[ ]: X_unlabeled = X_train.drop(index=labeled_idx) # Create a set keeping only the
     ↪ unlabeled data from X_train,
                                           # by dropping the indices of the
     ↪ labeled data
     y_unlabeled = np.full(len(X_unlabeled), -1) # Set the unlabeled dataset's target
     ↪ to -1 (no class)
```

```
[ ]: # Check correct distribution of labeled/unlabeled data
     (len(X_unlabeled) == 500) and (len(y_unlabeled) == 500)
```

True

```
[ ]: X_combined = pd.concat([X_labeled, X_unlabeled]) # pd.DataFrame
     y_combined = np.concatenate([y_labeled, y_unlabeled]) # np.array
```

Train the SelfTrainingClassifier on the combined labeled and unlabeled data

```
[ ]: self_training_clf.fit(X_combined, y_combined)

SelfTrainingClassifier(base_estimator=RandomForestClassifier(random_state=42),
                      threshold=0.99)
```

Train a fully supervised model only on the labeled data

```
[ ]: supervised_clf = RandomForestClassifier(random_state=42)
     supervised_clf.fit(X_labeled, y_labeled)
```

```
RandomForestClassifier(random_state=42)
```

Evaluate both models on the same test set

```
[ ]: y_pred_self_training = self_training_clf.predict(X_test)
     y_pred_supervised = supervised_clf.predict(X_test)
```

```
[ ]: print("\nSelf-Training Classifier Performance:")
     print(classification_report(y_test, y_pred_self_training))

     print("\nFully Supervised Classifier Performance:")
     print(classification_report(y_test, y_pred_supervised))
```

Self-Training Classifier Performance:

	precision	recall	f1-score	support
0	0.81	0.77	0.79	44
1	0.62	0.67	0.64	24
accuracy			0.74	68
macro avg	0.71	0.72	0.72	68
weighted avg	0.74	0.74	0.74	68

Fully Supervised Classifier Performance:

	precision	recall	f1-score	support
0	0.81	0.80	0.80	44
1	0.64	0.67	0.65	24
accuracy			0.75	68
macro avg	0.73	0.73	0.73	68
weighted avg	0.75	0.75	0.75	68

```
[ ]: # Convert classification reports into DataFrames
report_self_training = classification_report(y_test, y_pred_self_training,
↪output_dict=True)
report_supervised = classification_report(y_test, y_pred_supervised,
↪output_dict=True)

# Convert to Pandas DataFrame for better visualization
df_self_training = pd.DataFrame(report_self_training).transpose()
df_supervised = pd.DataFrame(report_supervised).transpose()
```

```
[ ]: df_self_training
```

	precision	recall	f1-score	support
0	0.809524	0.772727	0.790698	44.000000
1	0.615385	0.666667	0.640000	24.000000
accuracy	0.735294	0.735294	0.735294	0.735294
macro avg	0.712454	0.719697	0.715349	68.000000
weighted avg	0.741004	0.735294	0.737510	68.000000

```
[ ]: df_supervised
```

	precision	recall	f1-score	support
0	0.813953	0.795455	0.804598	44.00
1	0.640000	0.666667	0.653061	24.00
accuracy	0.750000	0.750000	0.750000	0.75
macro avg	0.726977	0.731061	0.728829	68.00

weighted avg 0.752558 0.750000 0.751114 68.00

The fully supervised model has slightly better performance across all metrics. It is better at identifying non-diabetic patients (Class 0), making fewer false negatives, and also makes slightly fewer false positives for diabetic patients (Class 1). However, self-training achieves nearly the same performance as full supervision, despite using only 200 labeled samples. This highlights its ability to perform competitively with minimal labeled data, making it a promising approach when labeled data is limited.